VERSION 1.5.0 · WRITTEN TO BE READ, NOT PRESENTED

PM-OS

A PM-led operating layer for the product development lifecycle.

It takes the context a product manager already has — research, a brief, a PRD, or a live codebase — and turns it into a chain of reviewable product documents, where every step is blocked until a human approves the one before it. It runs on the PM's own machine, inside the AI coding agent they already use. There is no app to log into.

Local-first | Human-gated | Claude Code + OpenAI Codex | 26 agent skills | Markdown · YAML · JSONL | No backend, no GUI

WHAT'S IN HERE — ABOUT A 10-MINUTE READ

01 Why it exists
02 What PM-OS is — and isn't
03 The pipeline
04 What a gate actually does
05 Three ways in
06 Changing a live product
07 The traceability spine
08 Keeping the documents honest
09 Getting it to the team
10 What the PM actually does
11 Shipped, and not
12 Start here

Product definition is slow, inconsistent, and ungated

Nothing below is a tooling complaint. These are three failure modes that show up on almost every product, and each one costs rework later in the build.

The thread breaks

Research, brief, scope, PRD, design and QA live in different documents and different tools. Decisions drift between them, and months later nobody can trace why a given requirement exists.

Quality is uneven

Every PM writes specs differently. What a developer receives depends on who wrote it and how much time that person had that week — not on any agreed standard.

Nothing is enforced

Work races ahead of approval. Design builds on a scope nobody signed off; QA writes tests against a PRD that has already changed underneath them.

> *The cost is rework, slow handoffs, and – in regulated work – real compliance risk. PM-OS attacks the third problem first, because fixing enforcement is what makes the other two stay fixed.*

A gated pipeline that turns context into approved artifacts

PM-OS is not an application and not a document generator. It is a **lifecycle state machine that the PM drives and an AI agent powers**. The agent drafts; the PM judges; the state machine refuses to move until the PM says so.

1

A skill suite

26 skills installed into the PM's existing AI agent. The PM types `/pm-new`, `/pm-stage-03-prd`, `/pm-approve` and the agent does the work. Nothing to deploy, no service to stand up.

2

Plain files

Every stage is Markdown, with state in YAML and an event log in JSONL — all on the PM's own machine. Portable, diffable, editable in any editor, and not locked inside a vendor's tool.

3

Human gates

Each stage is a draft until it is explicitly approved. The next stage physically cannot run until the one before it is approved — enforced by code, not convention.

4

The PM decides

The agent prepares and validates. The PM sets scope, makes trade-offs, and approves. Nothing progresses autonomously, and no stage self-approves.

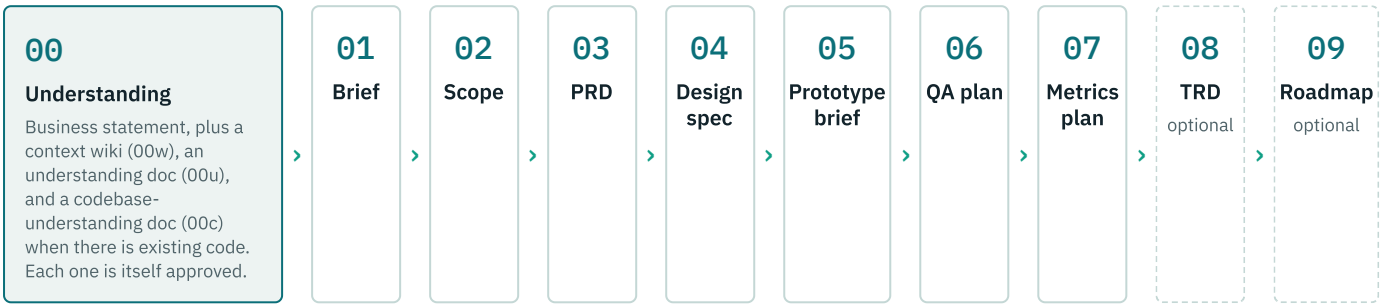
What it is not

Not a replacement for product judgment — it drafts and recommends, the PM decides. Not a system of record for live execution; Jira, Linear and Asana keep that job. Not a place for confidential customer data, PHI, PII, secrets or credentials. And not a sign-off authority — approving a stage records **your** decision, it does not substitute for whatever formal sign-off your organisation requires.

> **On the surface it presents:** *PM-OS deliberately has no graphical interface. The PM works in conversation with their agent and reviews Markdown files. That keeps the artifacts open and portable, and it means the tool inherits every improvement to the underlying agent.*

Understanding first, then seven gated stages

A project starts by building shared understanding, then walks a fixed sequence of stages. Each stage writes one Markdown document. Each arrow below is a **gate** — an approval the agent cannot cross on its own.



Company context overlay — company, team, glossary and guardrails, injected into every stage above. Installed once per machine, so artifacts speak your domain's vocabulary and respect your rules without the PM repeating them in every prompt.

› Each arrow is a human approval gate 08 / 09 Optional capstones, run after the core pipeline is approved

Later stages read approved upstream work

Stage 04 doesn't re-guess the requirements — it reads the approved PRD. Stage 06 tests what the PRD and design spec actually say. Approved upstream documents are the binding source of truth for everything below them, which is why the gates matter.

Two stages produce something you can click

Approving the design spec renders a browsable HTML preview. Approving the prototype brief generates a working interactive HTML prototype — real screens, real navigation — so reviewers can react to a thing rather than a description.

What a gate actually does

"Approval gate" sounds like process language. In PM-OS it is three concrete mechanical behaviours that run every time a stage starts.

Blocks unapproved work	A stage refuses to run while any stage above it is pending, still a draft, or stale. This is checked by code before generation begins, so it is not a guideline someone can decide to skip under deadline pressure.
Detects silent edits	Every approved document's body is hashed. If someone edits an approved document after the fact, the next gate re-hashes it, notices, and marks it **edited** — then asks the PM to re-approve before anything downstream is allowed to build on it. You cannot quietly rewrite a decision after sign-off.
Cascades staleness	Re-approve an upstream document and every already-approved document beneath it is marked **stale**. The PM sees exactly what now rests on a changed decision, instead of finding out during QA.

The five states a stage can be in

`pending` `draft` `approved` `edited` `stale`

`pending` not written yet · `draft` generated, awaiting review · `approved` signed off and binding · `edited` changed after approval · `stale` something above it moved. The last two are the honest ones: they exist so the project can admit it is out of sync.

> *Net effect: the document set can never lie about what has actually been reviewed.*

Three ways in — including "I already have all this"

A blank-page tool is useless to a PM who walks in with six months of research. PM-OS opens on one of three entry routes, and all three end up in the same gated pipeline.

ROUTE	USE IT WHEN	WHAT HAPPENS
`new`	**You're starting from an idea** A business statement, or a fuzzy problem.	Straight into stage 01. Or seed it first from material you already authored — research, a brief, a scope doc, a PRD, design notes. PM-OS reads them, builds a context wiki and an understanding doc for you to approve, adopts the documents you wrote, and back-fills the gaps beneath them.
`prototype`	**You have a prototype or design, but no code** Sales built a demo; design has a Figma flow.	The prototype is ingested as reviewed context *before* stage 01, so it enters as examined evidence rather than an unquestioned starting assumption.
`enhancement`	**You're changing a live product** The existing codebase is the ground truth.	Point PM-OS at the code — a Git URL, a local folder, or a `.zip`. It performs a strictly read-only scan and produces a codebase-understanding document that every downstream stage cites, so the brief describes the change rather than re-describing the whole product.

> **Why the import path matters:** *a PM's existing PRD is adopted, not regenerated. Re-deriving a document that already exists and is being built against would create a competing source of truth — the exact problem this tool is meant to remove.*

Changing a live product without rewriting its whole definition

Most product work isn't greenfield — it's the fourteenth change to something already in production. v1.5's headline capability handles that case inside the **same project**, so a product accumulates one continuous definition history instead of a graveyard of one-off docs.

`start`	**Freeze a baseline** The currently approved artifacts and the current state of the code are frozen into an immutable snapshot. That snapshot is what every later comparison is measured against.
`set-boundary`	**Decide what's in scope** A short interview records the *affected slice* — which parts of the product this change touches. Stages will not generate until this is recorded, which stops an "enhancement" from quietly becoming an unbounded rewrite.
`stages 01–09`	**Regenerate in place** The same pipeline runs, but only the affected sections are updated. Everything outside the boundary is carried forward untouched, and the gates still apply exactly as before.
`delta`	**See only what changed** PM-OS derives the change set between the frozen baseline and now: changed stories and epics, changed QA scenarios, new non-functional requirements, and explicit removals.
`complete`	**Close the cycle** The updated artifacts become the baseline for the next change. Two cycles of provenance are kept, so you can see what the definition looked like before.

Your code is never modified

The codebase is cloned or extracted into a read-only working copy inside the project and pinned to a commit. PM-OS reads it to understand the system; it never writes to it, and it warns the PM if the source has drifted since the scan.

Handoff scopes itself to the delta

The team receives the change, not the entire product definition again — the changed stories plus the minimum parent context needed to make sense of them, with removals stated explicitly rather than left implied.

Every requirement carries an ID, and the IDs connect

This is what separates PM-OS from a folder of well-written documents. Requirements, stories, screens, test cases and engineering tasks all carry stable identifiers, and the links between them are maintained automatically as stages are approved.

ID	WHAT IT NAMES	DECLARED IN
EPIC-####	A product workstream or outcome. Every story and requirement belongs to exactly one.	PRD
US-####	A user story, with its happy path and edge cases spelled out.	PRD
FR-#### / REQ-####	A functional requirement, carrying an explicit priority.	PRD
UJ-####	A user journey, flagged for whether the prototype should cover it.	PRD
SCR-####	A screen, declaring which stories and journeys it serves.	Design spec
TC-####	A test case, tied to the requirement it covers.	QA plan
TSK-####	An engineering task, tied to the requirement it implements.	TRD

Questions it can answer locally

Which test scenarios cover this requirement? Which screens does this story touch? Which backend tasks implement it? Which stories has no screen been designed for? All answered from the files, with no service to query.

It rebuilds itself on approval

The link table is regenerated whenever a stage is approved, and only *approved* stages contribute. A screen invented in a draft design spec can never leak into a developer's story file.

It survives into the tracker

When work is exported to Jira, each ticket carries its PM-OS identifier as a field and a label, and the created ticket keys are written back — so the chain still connects after the work leaves PM-OS.

The documents themselves are checked, not just approved

An approval gate confirms a human looked. It doesn't confirm the document is complete. So PM-OS also holds artifacts to a structural contract and can audit a whole project on demand.

Artifact contracts	Each stage has structural expectations: a PRD must declare journeys, epics, priorities and per-story happy paths and edge cases; a design spec must trace journeys to flows and declare its screens; a prototype brief must record which high-priority journeys it deliberately left out. Missing pieces **block generation** while the document is being written, and are reported as warnings at approval time so existing projects are never trapped.
A read-only project audit	One command answers "is this project internally consistent right now?" — checking state against document frontmatter, drift in approved bodies, the shape of upstream approvals, missing artifacts, duplicate or orphaned identifiers, and story coverage. It reports; it never silently repairs.
A tamper-evident event log	Every generation, approval and import appends a hash-chained event to the project's log — each entry carrying the hash of the one before it. It also records how long the PM took to approve and how much they edited the draft, which is how the team learns which stages the agent is actually good at.

> **Why the edit-distance measure matters:** *it is the only honest signal of draft quality. A stage the PM rewrites heavily every time is a stage whose prompt needs work — and that shows up as a number rather than a vague sense that "the PRDs aren't great."*

Getting it to the people who build it

A definition nobody reads is worthless. Export takes three shapes, and the one that gets used most is the middle one — because developers, designers and QA each want a different slice of the same approved material.

Raw text

One stage, or every approved stage, dumped as plain text. For pasting into an email, a ticket, or a chat thread. Deliberately dumb, and useful precisely because of it.

A per-audience package

The approved pipeline is decomposed into four folders — `dev`, `design`, `qa`, `business` — each carrying only what that audience needs. One file per story, plus reference material and the full source documents behind it.

Jira tickets

The approved definition becomes epics, stories and tasks: dry run, confirm, then create through the Atlassian connector, with ticket keys recorded back. No connector? An offline CSV for Jira's own importer ends in the same state.

What a developer opens

A single story file, containing the story itself and its requirements, its acceptance-level happy path and edge cases, **the screens it touches** — each one a direct link into the interactive prototype at that exact screen — **the test cases that cover it**, and **the backend tasks that implement it**. Where the source material genuinely doesn't say, the file says so plainly instead of inventing an answer.

> *Every section of that file is resolved from approved documents through the identifier spine. It is assembled, not re-written — so it cannot disagree with the PRD.*

What the PM actually does, in order

The whole loop is: **ask for a stage, read it, fix what's wrong, approve it.** Repeat seven times. The tool's job is to make the "fix what's wrong" step honest and the "approve it" step mean something.

`/pm-new`	**Create the project** Name it, say what it's for, and pick the entry route. If you have existing material, hand it over now rather than later.
`/pm-context-import`	**Give it your context** Point it at your research, documents, or codebase. It builds an understanding document that states what it learned, how confident it is in each part, and what it still doesn't know — and you approve that before any spec is written.
`/pm-interview`	**Close the open questions** The open unknowns stay listed. When you have answers, re-run the interview against just those gaps; your answers are recorded as high-confidence, PM-authored context and the resolved gaps are marked closed.
`/pm-stage-NN`	**Generate a stage** The gate checks upstream approvals first, then the agent writes the document using the approved upstream work and your company context. Deep-reasoning stages are flagged to run on the strongest model available.
`read & edit`	**Review it like a document** It's Markdown. Edit it directly, or ask the agent to change a section. Both are normal; re-approval absorbs the change and the hash follows it.
`/pm-approve`	**Approve, and unlock the next stage** Your decision is recorded, downstream staleness is cascaded, HTML companions are rendered where relevant, and the next stage becomes runnable.
`/pm-status` · `/pm-check`	**Know where you stand** Where every stage is, what changed recently, whether anything drifted, and how many known unknowns are still open.
`/pm-handoff`	**Ship the definition** Raw text, the per-audience package, or Jira tickets — scoped automatically to the delta when you're in an enhancement cycle.

> **Also there when you need them:** `/pm-feedback` *to rate a stage's output (this is how the prompts improve),* `/pm-enhance` *for the enhancement cycle,* `/pm-prototype-html` *to regenerate the clickable prototype, and* `/pm-sync` *to push telemetry and feedback to the team's shared repository.*

What's shipped, and what isn't yet

v1.5 is a complete **product-definition** system with one real external integration. The rest of the lifecycle is designed but not built. That distinction is worth being precise about, because the roadmap reads more impressive than the current tool if you blur it.

CAPABILITY	STATUS	DETAIL
Gated product definition	SHIPPED	Understanding group plus stages 01–07, with optional TRD and roadmap capstones.
Flexible intake	SHIPPED	Three entry routes; import and adopt existing PM-authored material; re-runnable discovery interview.
Enhancement cycles	SHIPPED	Frozen baselines, recorded affected slice, read-only codebase understanding, delta-only handoff.
Traceability spine	SHIPPED	Epics, stories, requirements, journeys, screens, test cases and tasks, with a local resolver.
Artifact contracts & project audit	SHIPPED	Structural validation at generation and approval; read-only consistency check.
Per-audience handoff package	SHIPPED	Decomposed dev / design / QA / business folders with full source projections.
Jira export	SHIPPED	Via the Atlassian connector, or a fully offline CSV for Jira's own importer.
Two agent runtimes	SHIPPED	Claude Code and OpenAI Codex, with verified gate parity. Offline and mirrored installs supported.
Delivery model Scope tiers and delivery increments	DESIGNED	Planned and specified, not yet implemented.
Design integration Figma pull and push	DESIGNED	The design half of external handoff. A pilot is underway to move design authority to the designer; productization is planned.
Linear export · Gemini runtime	PLANNED	The extension points exist; the integrations don't.
Dev-phase support	NOT BUILT	Connecting requirements to pull requests and in-flight implementation.
QA bug triage	NOT BUILT	Analysing reported bugs against the codebase and suggesting fix plans.
Release readiness · feedback loop	NOT BUILT	Release decisions, and ingesting shipped-product feedback back into the next cycle.

> *The direction is the same gated, PM-led model widened across the lifecycle – intake, definition, design, dev handoff, QA, release, feedback – consuming artifacts from wherever work actually happens. The PM stays the decision-maker at every new stage.*

Trying it takes about ten minutes

You need **Git, Python 3.11 or newer, and one of the two supported agents**. Everything installs into your home directory; nothing runs as a service and nothing calls home.

1 Install

Clone the repository and run the installer for your agent:

`./install.sh --runtime claude`

On a locked-down corporate machine, an offline zip install and a GitLab mirror are both supported — see the offline install guide.

2 Verify

The installer checks itself: config, shared libraries, gate hooks, installed skills, and a self-test that runs the real gate in a throwaway project to confirm it blocks an unapproved stage.

If that self-test passes, the install is sound.

3 Run one project end to end

Pick something real but small. Create it, import whatever context you already have, and walk stages 01 to 07, approving each one.

Then run the handoff package and give it to a developer. Their reaction is the actual evaluation.

Where to read more

The operating procedure — `docs/guides/sop.md` — how, when and where to use PM-OS: roles, the standard workflow, best-fit cases, the pitfalls, and data-handling rules. Start here if you're deciding whether to adopt it.

The state of the product — `docs/roadmap/current-state-review.md` — the canonical review of what is implemented versus planned, with evidence for each claim. `docs/roadmap/backlog.md` tracks verified-but-unfixed issues.

How it's built — `ARCHITECTURE.md` — component wiring, the stage state machine, and the generate-then-approve sequence. `docs/reference/generated-doc-formats.md` shows the section-by-section shape of every document PM-OS produces.

> **One question worth asking a PM who has used it:** *not "did it write a good PRD?" but "did anything downstream get built on a decision nobody had approved?" That's the problem this tool was built to make impossible.*